Student: Molapo Setsumi
Student Number: 202322042

1. Purpose and Scope

This System Design Document translates the system requirements into a design for the Moderated Learning Management System (MLMS). The system is a menu-driven C++17 application using a local SQLite database. It operates on one computer without requiring an internet connection and supports Administrators, Lecturers, Heads of Department (HoD), and Students.

The design covers the architecture and components, UML class and behavioural models, design patterns, database structure, and the controls needed to support assignment submission, marking, moderation, and controlled release of marks.

2. Requirements Baseline and Assumptions

The design is based on the Week 2 Software Requirements Specification (SRS). The table below identifies the main links between requirements and the design elements. Requirement identifiers should be checked against the final approved SRS before implementation.

SRS area	Design response
FR-01 / FR-02; NFR-03 / NFR-04	Authentication service, password-hash abstraction, account state, and central permission checks.
FR-03 to FR-07	User, department, course, lecturer allocation, enrolment, and assignment entities and persistence.
FR-08 to FR-10	Submission/version model, effective-deadline calculation, extensions, and late-penalty policy.
FR-11 to FR-13	Marking, moderation decisions, release states, and student visibility controlled by release status.
FR-14 / FR-18; NFR-06	Audit records for mark-related changes and role-scoped audit queries.
FR-15 / FR-16; NFR-05	Lifecycle rules, input validation, foreign keys, and transactional writes.
NFR-01 / NFR-02	Performance targets and offline operation.
NFR-08 to NFR-10	C++17/CMake structure, backup/restore expectations, and Windows/Linux installation considerations.

2.1 Policy assumptions and unresolved decisions

- The effective deadline is the original assignment deadline unless a student-specific extension supplies a later deadline.
- The penalty is 10% of the assignment maximum mark per started 24-hour period after the effective deadline, with the final mark floored at zero. Exactly 24 hours late is day one.
- The SRS states that late work is accepted for up to three days and work submitted more than three days late receives zero. The UC-01 exception flow also says an attempt beyond three days is recorded for audit and feedback. The exact retention rule for submission content versus attempt metadata must be confirmed with the supervisor.
- Resubmission is permitted only before the effective deadline and while the submission is unmarked. Earlier versions are retained, and the latest valid version is selected for marking.
- A mark may be released only after HoD approval. Feedback is released with its mark.
- The design baseline allows one extension per student/assignment pair. Whether an extension can be changed or multiple extensions can be granted remains to be confirmed.
- The SRS prefers Argon2id password hashing and refers to account lockout after repeated failed logins, but does not define the attempt threshold, lockout duration, or reset behaviour. These values must be agreed before implementation and testing.

2.2 Design boundary

This document describes a proposed design. It does not claim that the software has already been implemented or tested. Implementation-specific details may be refined only where they remain consistent with the approved SRS.

3. Architecture and Component Design

3.1 Selected architecture

A layered architecture is selected because it separates user interaction from application workflows, business rules, and SQLite persistence. This supports maintainability and permits domain rules to be tested independently of the console and database. The console layer does not execute SQL directly.

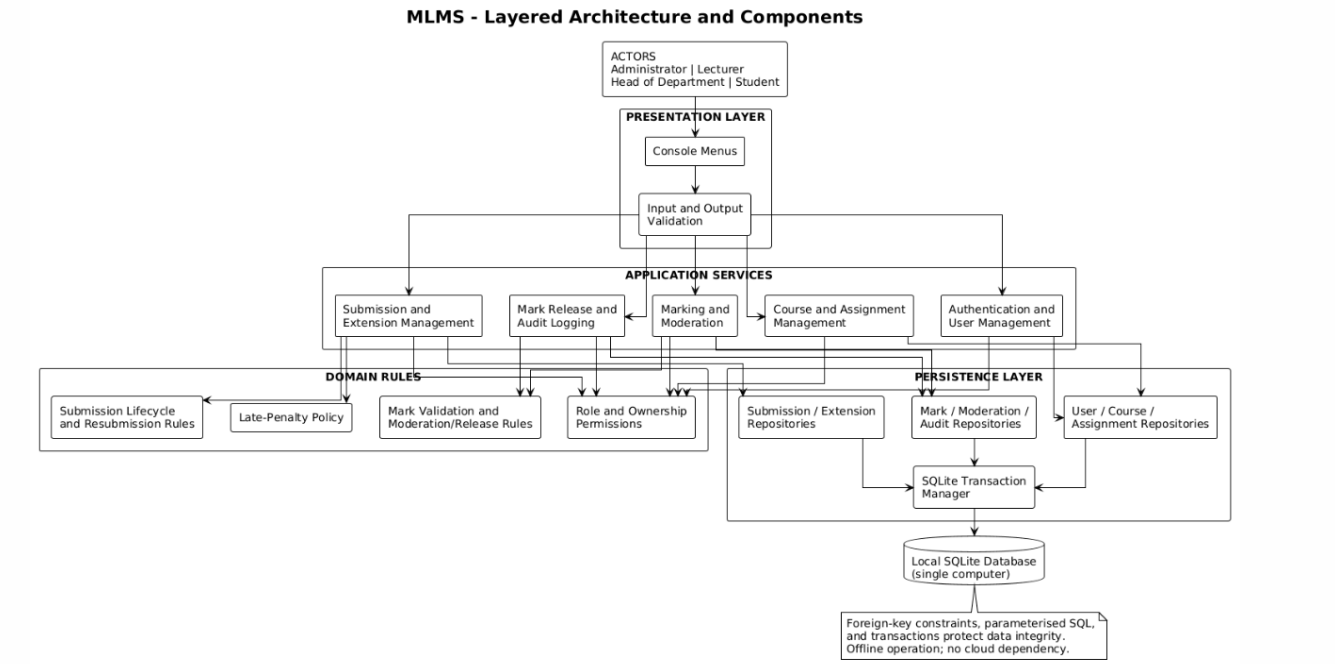


Figure 1. Proposed architecture and component boundaries.

3.2 Layer responsibilities

Layer	Responsibilities	Design constraint
Presentation / console	Display role-specific menus, collect input, format results/errors, confirm destructive or release actions.	No direct SQL and no reliance on menu visibility as the only access control.
Application services	Coordinate authentication, administration, assignments, submissions, marking, moderation, release, and audit queries.	Authorise operations, validate workflow preconditions, and coordinate transactions.
Domain / business rules	Represent entities and enforce deadline, lifecycle, mark-range, penalty, and visibility rules.	Rules remain deterministic and testable without console or SQLite dependencies.
Repository / DAO	Encapsulate parameterised SQL and map rows to domain objects.	Persistent access passes through repositories; use foreign keys and transactions.
SQLite persistence	Store accounts, courses, assignments, submission versions, extensions, marks, moderation decisions, and audit records.	Local database; foreign keys enabled; backups supported by documented procedures.

3.3 Main components

Component	Primary responsibility
AuthenticationService	Verifies credentials using an approved password-hashing library, establishes the authenticated context, rejects inactive/locked accounts, and tracks failed attempts.
AuthorizationService / PermissionPolicy	Checks role permissions and resource scope, including assigned courses, department ownership, and a student's own records.
UserManagementService	Creates, updates, deactivates, and reactivates accounts and maintains role assignments.
CourseManagementService	Maintains departments, courses, lecturer-course allocations, and student enrolments.
AssignmentService	Creates assignments, validates titles/deadlines/maximum marks, and lists assignments for enrolled students.
ExtensionService	Grants a student-specific revised deadline with a required reason and exposes it to the student and relevant HoD.
SubmissionService	Accepts eligible submissions, preserves versions, enforces deadline/resubmission rules, and records submission status.

LatePenaltyPolicy	Calculates started late days and the final mark according to the approved late policy.
MarkingService	Validates marks and feedback and saves the assessment awaiting moderation.
ModerationService	Allows an authorised HoD to approve, amend with a reason, or return a mark.
ReleaseService	Releases only approved marks and their feedback in a transaction.
AuditService	Creates and queries audit records, subject to role and department scope.
Repositories / TransactionManager	Persist and retrieve records through parameterised SQLite statements and coordinate commit/rollback.

3.4 Dependency rules and rejected alternative

- Presentation calls application services; it does not call repositories directly.
- Application services use domain rules and repository abstractions. Domain rules do not depend on console code or SQLite.
- Repositories depend on the database connection and transaction abstraction.
- Mark entry, moderation changes, penalty recalculation, and release must commit with their required audit records or roll back together.
- Student result queries filter by the authenticated student's ID and Released status. Hiding a menu item is not sufficient authorisation.

An alternative considered was Model–View–Controller (MVC). MVC is useful when an application has a more complex user interface or multiple views. This system has a relatively simple menu-driven console, so a layered architecture provides sufficient separation between presentation, application services, business rules, and persistence without introducing unnecessary structural complexity.

4. Class Design, Design Patterns and SOLID

4.1 Core domain class model

The class model represents the principal entities and abstractions used by the system. Visibility, attribute/operation types, inheritance or interfaces, and multiplicities should be read together with the class diagram.

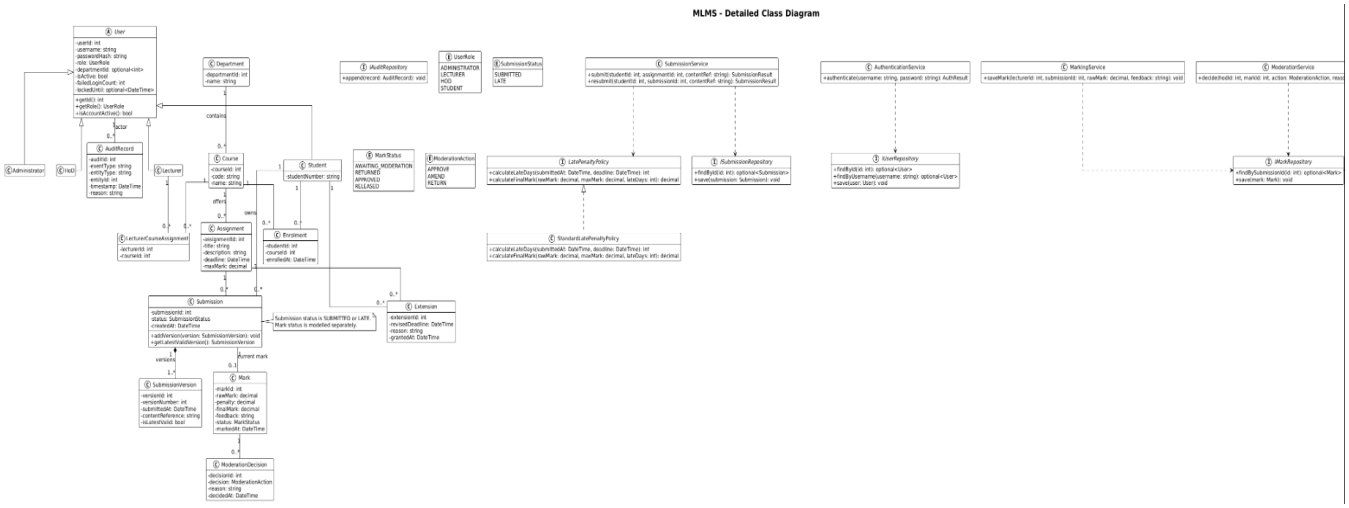


Figure 2. Detailed UML class model.

4.2 Principal classes and responsibilities

Class	Key data / operations	Responsibility
User (abstract)	userId, username, passwordHash, role, active, departmentId, failedLoginCount, lockedUntil; getRole()	Common identity and account state. Plaintext passwords are never stored.
Administrator / Lecturer / HoD / Student	Role-specific identity; student number for Student	Represent the four SRS roles. Resource checks remain centralised in permission services.
Department	departmentId, name	Owns courses and defines HoD scope.
Course	courseId, code, name, departmentId	Belongs to one department.
LecturerCourseAssignment	lecturerId, courseId	Represents lecturer/course many-to-many allocation.
Enrolment	studentId, courseId, enrolledAt	Represents student/course enrolment.
Assignment	assignmentId, courseId, title, description, deadline, maxMark	Defines an assessment and its original deadline.

Submission	submissionId, assignmentId, studentId, status; addVersion()	Represents one logical student/assignment submission and links to its version history.
SubmissionVersion	versionId, submissionId, submittedAt, contentReference, isLatestValid	Preserves accepted versions and identifies the version selected for marking.
Extension	extensionId, assignmentId, studentId, revisedDeadline, reason, grantedBy, grantedAt	Stores an individual revised deadline and its rationale.
Mark	markId, submissionId, rawMark, penalty, finalMark, feedback, status	Stores assessment results and release status; feedback is hidden until release.
ModerationDecision	decisionId, markId, HoD, decision, reason, decidedAt	Records approval, amendment, or return decisions.
AuditRecord	auditId, actorId, eventType, timestamp, entityId, oldValue, newValue, reason	Provides an append-only application-level history of mark-related actions.
LatePenaltyPolicy	calculateLateDays(), calculateFinalMark()	Pure policy abstraction with no database or UI dependency.

4.3 Relationships and multiplicities

- One Department contains many Courses; each Course belongs to one Department.
- Lecturers and Courses have a many-to-many relationship represented by LecturerCourseAssignment.
- Students and Courses have a many-to-many relationship represented by Enrolment.
- One Course has many Assignments; one Assignment can have student Submissions.
- One Submission has one or more SubmissionVersions. At most one version is designated as the latest valid version for marking.
- The baseline model allows one current Mark per Submission; audit records preserve prior values when the mark changes.
- A Mark may have multiple ModerationDecision records across return-and-review cycles.
- The baseline schema allows at most one Extension per student/assignment pair. If the policy changes, the schema and effective-deadline rule must change consistently.

4.4 Design patterns

Each pattern below states its name, the problem it addresses, the proposed solution, and the consequences of using it.

Pattern 1 — Strategy

Problem description: Late-penalty rules are business policy and may change. Embedding the calculation directly in SubmissionService would couple the workflow to one policy and make the calculation harder to test independently.

Solution: Define a LatePenaltyPolicy interface and provide a StandardLatePenaltyPolicy implementation. The implementation calculates started 24-hour periods, applies 10% of the assignment maximum per period up to three days, and returns a zero final mark beyond the permitted window, subject to the final approved SRS policy.

Consequences: The calculation is isolated, replaceable, and straightforward to unit-test. The design adds an interface and requires any replacement policy to satisfy the same contract and approved SRS rules.

Pattern 2 — Repository

Problem description: If application services execute SQLite statements directly, business workflows become coupled to persistence details and are harder to test.

Solution: Define repository abstractions such as IUserRepository, ISubmissionRepository, IMarkRepository, and IAuditRepository, with SQLite-specific implementations. Application services depend on the abstractions rather than SQL statements.

Consequences: Services can be tested with fake repositories and persistence details are isolated. The trade-off is additional mapping code and a need to coordinate transactions across repositories.

Pattern 3 — Dependency Injection

Problem description: Services that construct their own repositories, penalty policy, or transaction manager are tightly coupled to concrete implementations.

Solution: Supply repository, policy, and transaction dependencies to services through their constructors.

Consequences: Dependencies can be substituted for tests and implementations can change with less service-level modification. The application composition code must configure the dependencies correctly.

Pattern 4 — Factory (conditional)

Problem description: Constructing role-specific User objects in multiple places can duplicate role-selection and construction logic.

Solution: If the UML uses distinct User subtypes, a UserFactory can centralise creation from validated role data. Password hashing and authentication remain separate responsibilities.

Consequences: Construction is consistent and centralised, but the factory must be updated when a new subtype is added. Keep this pattern only if the implementation genuinely needs role-specific polymorphic objects; a role field alone does not require a factory.

4.5 SOLID application

Principle	Application in this design
S — Single Responsibility	SubmissionService coordinates submission workflow; LatePenaltyPolicy calculates penalties; repositories handle persistence; AuditService records audit events.
O — Open/Closed	A penalty strategy can be replaced without rewriting SubmissionService, provided it follows the same policy interface and approved rules.
L — Liskov Substitution	Any concrete User subtype must be usable wherever User is expected without weakening account and authorisation invariants.
I — Interface Segregation	Small interfaces such as ISubmissionRepository and IAuditRepository are preferred over one large interface containing unrelated operations.
D — Dependency Inversion	Application services depend on repository and policy abstractions; SQLite-specific repositories implement those abstractions.

5. Behavioural Models

5.1 Submission and mark lifecycle state machine

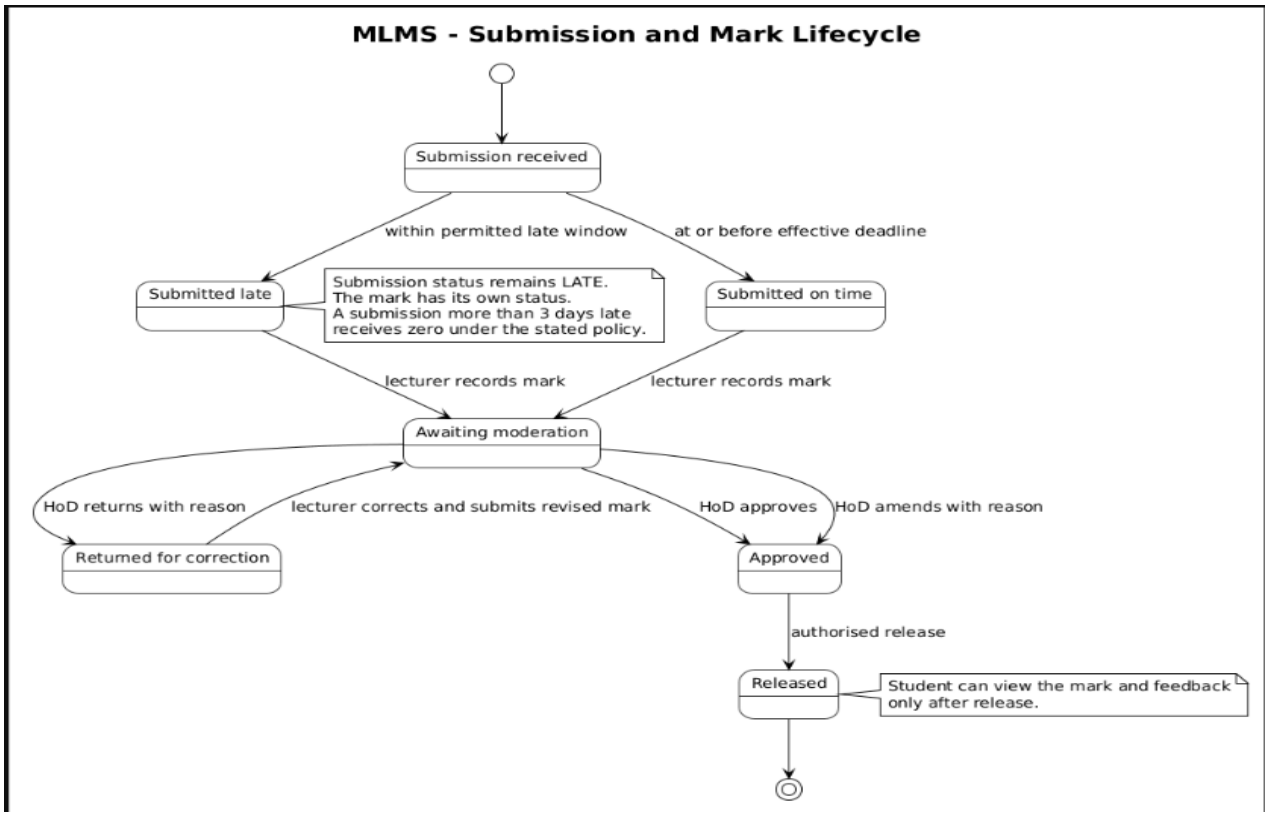


Figure 3. Submission and associated mark lifecycle.

The submission's status and the mark's moderation status are separate concerns. A submission remains SUBMITTED or LATE, while its associated mark moves through moderation and release. A HoD may approve, amend with a reason,

or return a mark; only an approved mark may be released. The student cannot access a mark or feedback before release.

Current state	Event / guard	Next state	Required side effect
No submission / Draft	Valid submission at or before effective deadline	Submitted	Save version and timestamp.
No submission / Draft	Accepted initial submission after deadline but within permitted window	Late	Save version and record lateness.
Submitted or Late	Effective deadline has passed and authorised lecturer marks	Mark awaiting moderation	Save mark/feedback and audit entry transactionally.
Awaiting moderation	HoD returns with reason	Returned	Save moderation decision and audit event.
Returned	Lecturer corrects/re-marks after deadline	Awaiting moderation	Save revised mark and audit change with reason.
Awaiting moderation	HoD approves or amends with reason	Approved	Save decision and audit any amendment.
Approved	HoD releases approved mark	Released	Change visibility status and record release audit in one transaction.
Any unreleased state	Student requests result	No transition	Deny mark/feedback access.

5.2 Swimlane activity model

Figure 4. End-to-end activity flow across Student, Lecturer, and HoD/System.

The activity model shows the flow from student submission through lecturer marking, HoD moderation, and controlled release. Decision points must enforce the effective deadline, lecturer assignment, department scope, reasons for moderation changes, and approval before release.

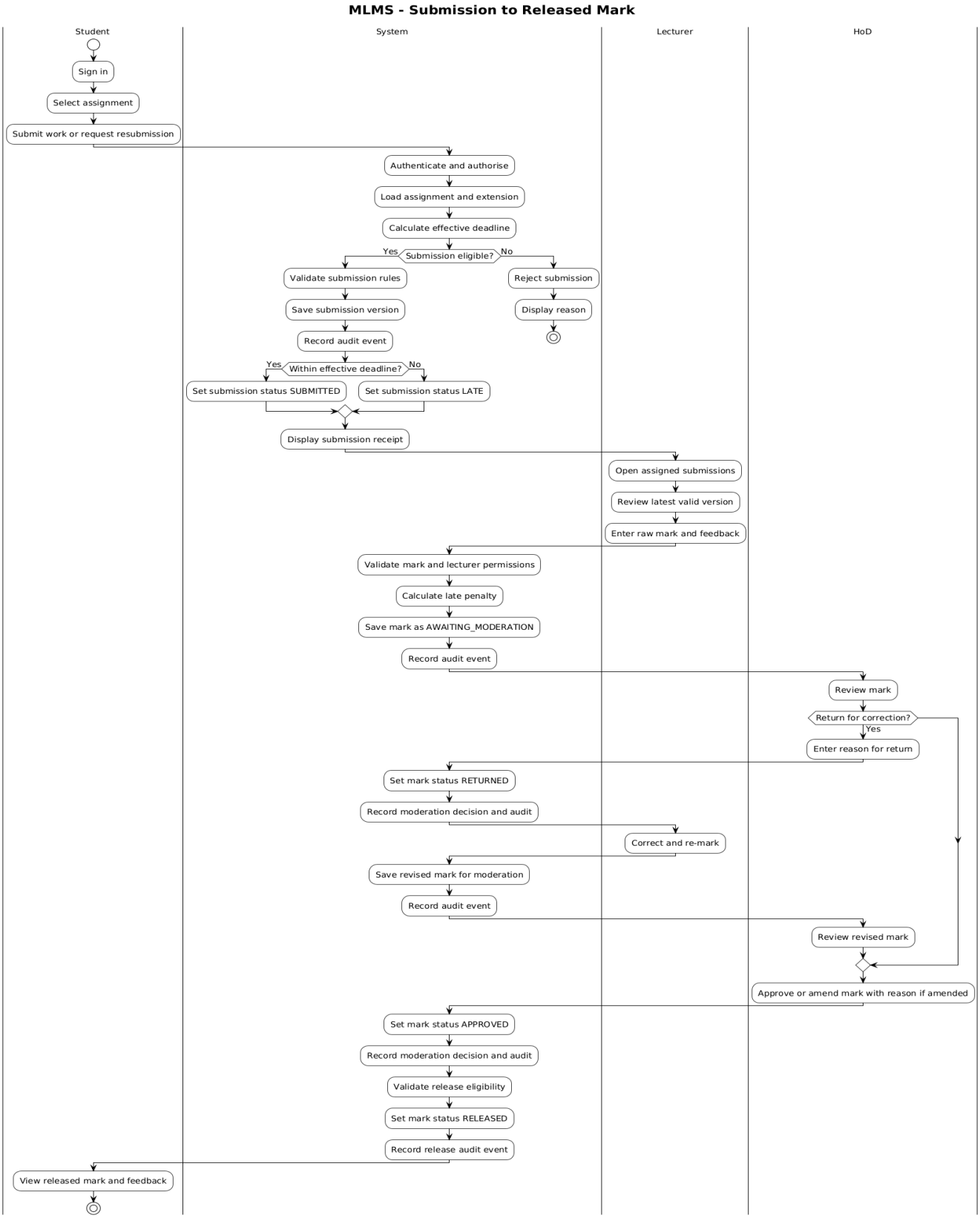


Figure 5. Submission sequence, including validation before persistence.

5.3 Sequence diagram — Submit assignment

The service obtains the effective deadline from the assignment and any applicable extension, checks enrolment and lifecycle eligibility, and creates a new version. For resubmission, the previous version remains stored. If persistence

fails, the transaction is rolled back and the console must not display a success receipt. The diagram should make the late-submission path and resubmission path explicit.

5.4 Sequence diagram — moderate and release marks

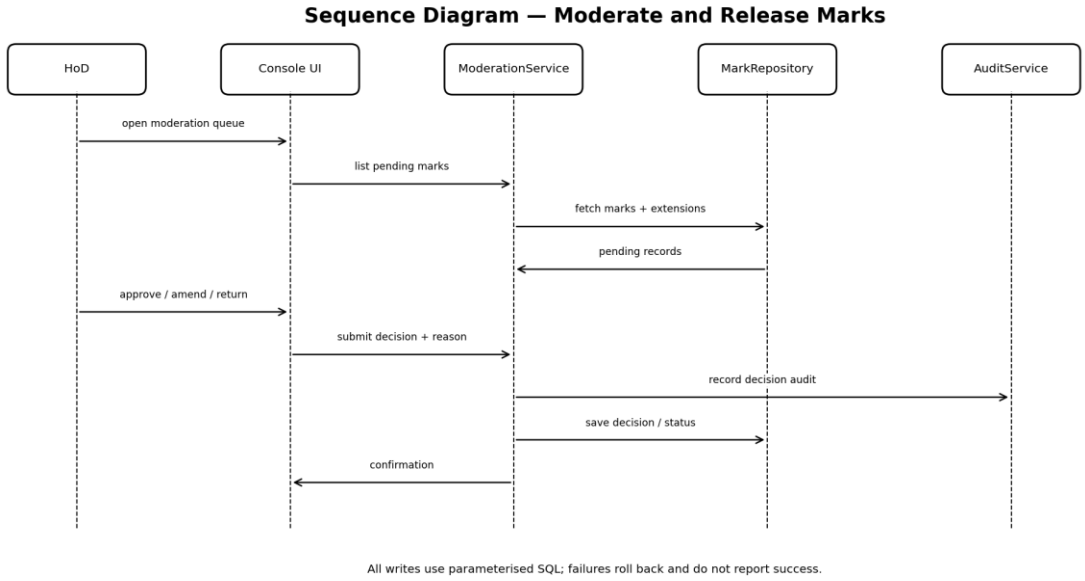


Figure 6. Moderation and release sequence.

Moderation is restricted to the HoD's department. A change or return requires a reason. Release checks every selected mark and uses one transaction for the release batch. An unapproved mark must not become visible even if a caller bypasses the menu.

5.5 Critical design invariants

- Student result queries return only records where the mark status is RELEASED and the submission belongs to the authenticated student.
- A lecturer may mark only a course to which they are assigned and only after the effective deadline has passed.
- An HoD may moderate or release only courses within their department.
- A mark change, moderation adjustment, penalty recalculation, or release and its required audit event commit together or roll back together.
- The final mark cannot be negative or exceed the assignment maximum; the penalty cannot produce a negative result.
- Resubmission is rejected after the effective deadline or once marking has started; an initial late submission follows the separate permitted late-window rule.
- A mark may enter RELEASED status only from APPROVED status.

6. Database Design

6.1 Database design goals and normalisation

The system uses relational SQLite tables for users, departments, courses, relationships, assignments, submissions, versions, extensions, marks, moderation decisions, and audit records. Separating these concepts avoids storing repeated course, user, submission-version, and moderation information in one large table. The design aims for Third Normal Form (3NF): each non-key attribute depends on the key, the whole key, and nothing but the key. Primary keys, foreign keys, unique constraints, and check constraints protect data integrity.

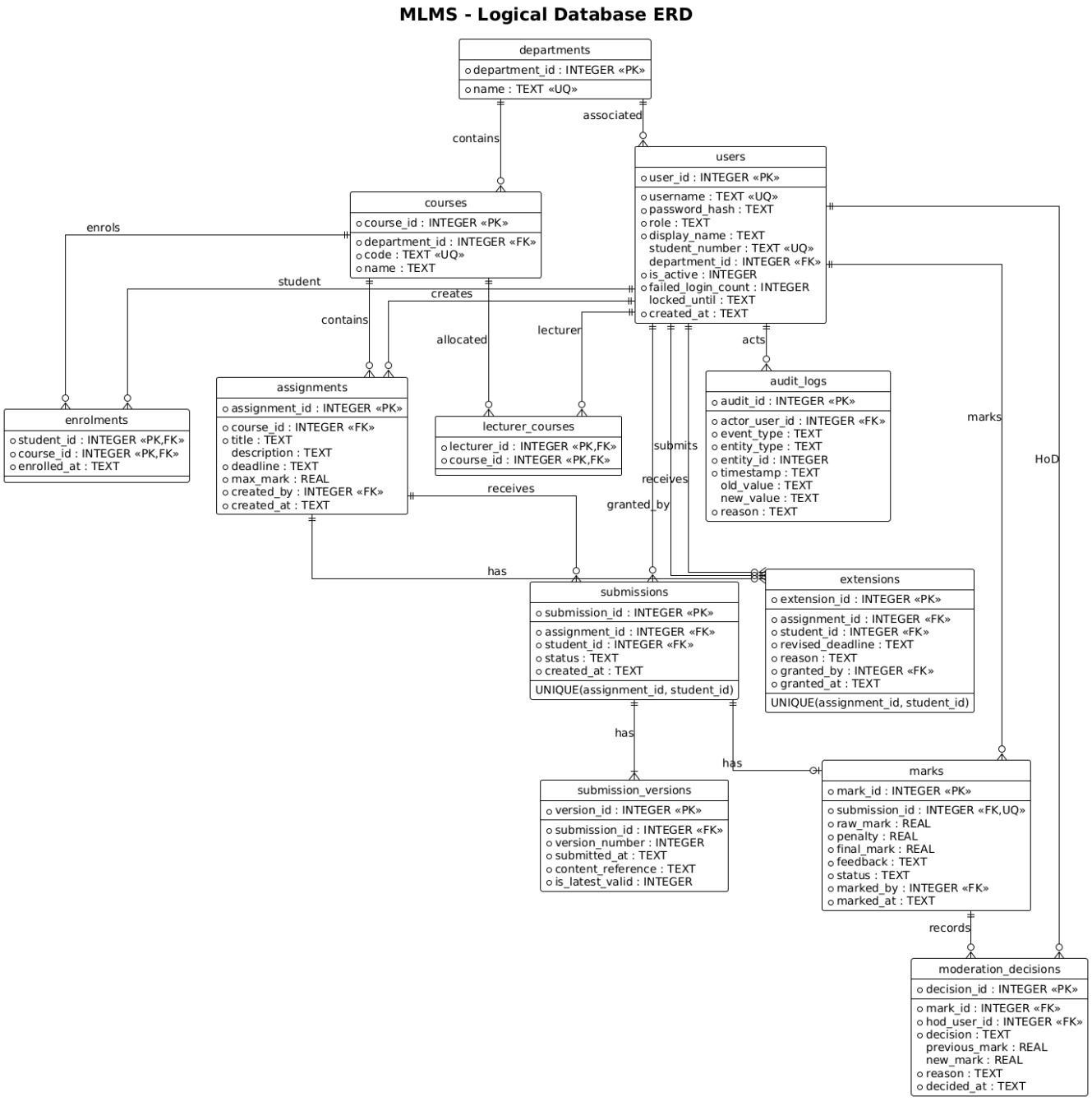


Figure 7. Logical entity-relationship diagram (ERD).

6.2 Main tables and keys

Table	Primary key	Purpose / key relationships
departments	department_id	Department names; parent of courses.
users	user_id	Account details, role, password hash, account status, and lockout fields.
courses	course_id	Course details; department_id references departments.
lecturer_courses	(lecturer_id, course_id)	Lecturer/course allocation; both columns reference users/courses.
enrolments	(student_id, course_id)	Student/course enrolment; both columns reference users/courses.
assignments	assignment_id	Assessment definition; course_id references courses.
submissions	submission_id	One logical submission per student/assignment pair.
submission_versions	version_id	Accepted versions and timestamps; submission_id references submissions.
extensions	extension_id	Student-specific revised deadline, reason, grantor, and grant time.
marks	mark_id	One current mark per submission in the baseline model.
moderation_decisions	decision_id	Moderation history, decision, reason, and timestamp.

audit_logs	audit_id	Actor, event, timestamp, entity reference, old/new values, and reason.

6.3 Data dictionary

Table.column	Type / constraint	Meaning
departments.department_id	INTEGER PRIMARY KEY	Internal department identifier.
departments.name	TEXT NOT NULL UNIQUE	Unique department name.
users.user_id	INTEGER PRIMARY KEY	Internal account identifier.
users.username	TEXT NOT NULL UNIQUE	Unique login name.
users.password_hash	TEXT NOT NULL	Password-hashing library output; never plaintext.
users.role	TEXT NOT NULL CHECK	ADMINISTRATOR, LECTURER, HOD, or STUDENT.
users.display_name	TEXT NOT NULL	Name displayed by the application.
users.department_id	INTEGER REFERENCES departments	Department association where applicable.
users.student_number	TEXT UNIQUE	Student number where applicable.
users.is_active	INTEGER NOT NULL DEFAULT 1 CHECK IN (0,1)	Whether the account may authenticate.
users.failed_login_count	INTEGER NOT NULL DEFAULT 0 CHECK >= 0	Number of consecutive/recorded failed login attempts according to the implemented policy.
users.locked_until	TEXT NULL	Timestamp until which login is blocked, if locked.
users.created_at	TEXT NOT NULL	Account creation timestamp.
courses.course_id	INTEGER PRIMARY KEY	Internal course identifier.
courses.department_id	INTEGER NOT NULL FK	Owning department.
courses.code	TEXT NOT NULL UNIQUE	Unique course code.
courses.name	TEXT NOT NULL	Course name.
lecturer_courses.lecturer_id	INTEGER FK; composite PK	Lecturer assigned to the course.
lecturer_courses.course_id	INTEGER FK; composite PK	Course in the lecturer allocation.
enrolments.student_id	INTEGER FK; composite PK	Enrolled student.
enrolments.course_id	INTEGER FK; composite PK	Course in which the student is enrolled.
enrolments.enrolled_at	TEXT NOT NULL	Enrolment timestamp.
assignments.assignment_id	INTEGER PRIMARY KEY	Internal assignment identifier.
assignments.deadline	TEXT NOT NULL	Assignment deadline in the agreed timestamp format.
assignments.max_mark	REAL NOT NULL CHECK > 0	Maximum possible mark.
assignments.created_by	INTEGER NOT NULL FK	User who created the assignment.
submissions.submission_id	INTEGER PRIMARY KEY	Logical submission identifier.
submissions.assignment_id / student_id	INTEGER NOT NULL FK; UNIQUE pair	Assignment and student; one logical submission per pair.
submissions.status	TEXT NOT NULL CHECK	SUBMITTED or LATE; separate from mark moderation status.
submission_versions.version_id	INTEGER PRIMARY KEY	Version identifier.
submission_versions.submission_id	INTEGER NOT NULL FK	Parent logical submission.
submission_versions.version_number	INTEGER NOT NULL; UNIQUE with submission_id	Version order within a submission.
submission_versions.submitted_at	TEXT NOT NULL	Recorded submission timestamp.
submission_versions.content_reference	TEXT NOT NULL	Reference to the stored submission content.
submission_versions.is_latest_valid	INTEGER NOT NULL DEFAULT 1 CHECK IN (0,1)	Whether the version is currently selected as latest valid version.
extensions.revised_deadline	TEXT NOT NULL	Student-specific effective deadline.
extensions.reason	TEXT NOT NULL; non-empty CHECK	Mandatory extension rationale.
extensions.granted_by / granted_at	FK / TEXT NOT NULL	Granting user and timestamp.
marks.submission_id	INTEGER NOT NULL UNIQUE FK	Submission associated with the current mark.
marks.raw_mark	REAL NOT NULL CHECK >= 0	Mark before late penalty; must not exceed assignment maximum.
marks.penalty	REAL NOT NULL DEFAULT 0 CHECK >= 0	Penalty applied under the approved policy.
marks.final_mark	REAL NOT NULL CHECK >= 0	Final result; must not exceed assignment maximum.
marks.feedback	TEXT NOT NULL DEFAULT ''	Feedback visible only after release.
marks.status	TEXT NOT NULL CHECK	AWAITING_MODERATION, RETURNED, APPROVED, or RELEASED.
marks.marked_by / marked_at	FK / TEXT NOT NULL	Lecturer and marking timestamp.
moderation_decisions.decision	TEXT CHECK	APPROVE, AMEND, or RETURN.
moderation_decisions.reason	TEXT NOT NULL	Reason for moderation action.
moderation_decisions.decided_at	TEXT NOT NULL	Decision timestamp.
audit_logs.actor_user_id	INTEGER NOT NULL FK	User who caused the event.
audit_logs.event_type / timestamp	TEXT NOT NULL	Event type and timestamp.
audit_logs.entity_type / entity_id	TEXT / INTEGER NOT NULL	Type and identifier of affected entity.
audit_logs.old_value / new_value	TEXT NULL	Before/after values where applicable.
audit_logs.reason	TEXT NOT NULL	Reason for change or defined reason for an automatic event.
assignments.course_id	INTEGER NOT	Course to which the assignment

	NULL FK	belongs.
assignments.title	TEXT NOT NULL	Assignment title.
assignments.description	TEXT NOT NULL DEFAULT ''	Assignment description; may be empty.
assignments.created_at	TEXT NOT NULL	Timestamp when the assignment was created.
submissions.created_at	TEXT NOT NULL	Timestamp when the logical submission record was created.
extensions.extension_id	INTEGER PRIMARY KEY	Internal extension identifier.
extensions.assignment_id / student_id	INTEGER NOT NULL FK; UNIQUE pair	Assignment and student receiving the extension; only one extension per pair in the baseline model.
marks.mark_id	INTEGER PRIMARY KEY	Internal identifier for the current mark record.
moderation_decisions.decision_id	INTEGER PRIMARY KEY	Internal identifier for a moderation decision.
moderation_decisions.mark_id	INTEGER NOT NULL FK	Mark associated with the moderation decision.
moderation_decisions.hod_user_id	INTEGER NOT NULL FK	HoD who made the moderation decision.
moderation_decisions.previous_mark / new_mark	REAL NULL	Mark values before and after an amendment, where applicable.

6.4 Class-to-table mapping

Domain class	Table(s)	Mapping rationale
User / Student / Lecturer / HoD / Administrator	users	Single-table representation; role distinguishes the account type. A separate profile table can be introduced only if role-specific fields require it.
Department	departments	One row per department.
Course	courses	department_id is a foreign key.
LecturerCourseAssignment	lecturer_courses	Composite primary key prevents duplicate allocations.
Enrolment	enrolments	Composite primary key prevents duplicate enrolments.
Assignment	assignments	course_id links the assignment to its course.
Submission	submissions	One logical row per student/assignment pair.
SubmissionVersion	submission_versions	One-to-many child table preserves prior versions.
Extension	extensions	Student-specific extension linked to assignment and student.
Mark	marks	One current mark row per submission; audit logs preserve change history.
ModerationDecision	moderation_decisions	Separate table preserves repeated moderation decisions.
AuditRecord	audit_logs	Append-only event table at application level.

The object model and relational model need not have identical shapes. Inheritance is represented by a role column in users, many-to-many associations use join tables, and submission versions and moderation decisions use child tables because the database must preserve multiple historical records.

6.5 Proposed SQL schema

The following schema is the design baseline. Enable foreign-key enforcement on every SQLite connection. Application validation must also enforce rules that cannot be expressed as a simple table CHECK constraint, such as comparing a mark with the maximum mark stored in another table.

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL UNIQUE
);

CREATE TABLE users (
    user_id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    department_id INTEGER REFERENCES departments(department_id),
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL CHECK (role IN ('ADMINISTRATOR','LECTURER','HOD','STUDENT')),
    display_name TEXT NOT NULL,
    student_number TEXT UNIQUE,
    is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
    failed_login_count INTEGER NOT NULL DEFAULT 0 CHECK (failed_login_count >= 0),
    locked_until TEXT,
    created_at TEXT NOT NULL
);

CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    department_id INTEGER NOT NULL REFERENCES departments(department_id),
    code TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL
);

CREATE TABLE lecturer_courses (
    lecturer_id INTEGER NOT NULL REFERENCES users(user_id),
    course_id INTEGER NOT NULL REFERENCES courses(course_id),
    PRIMARY KEY (lecturer_id, course_id)
);

CREATE TABLE enrolments (
    student_id INTEGER NOT NULL REFERENCES users(user_id),
    course_id INTEGER NOT NULL REFERENCES courses(course_id),
    enrolled_at TEXT NOT NULL,
    PRIMARY KEY (student_id, course_id)
);

CREATE TABLE assignments (
    assignment_id INTEGER PRIMARY KEY,
    course_id INTEGER NOT NULL REFERENCES courses(course_id),
    title TEXT NOT NULL,
    description TEXT NOT NULL DEFAULT '',
    deadline TEXT NOT NULL,
    max_mark REAL NOT NULL CHECK (max_mark > 0),
    created_by INTEGER NOT NULL REFERENCES users(user_id),
    created_at TEXT NOT NULL
);

CREATE TABLE submissions (
    submission_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL REFERENCES assignments(assignment_id),
    student_id INTEGER NOT NULL REFERENCES users(user_id),
    status TEXT NOT NULL CHECK (status IN ('SUBMITTED','LATE')),
    created_at TEXT NOT NULL,
    UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission_versions (
    version_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL REFERENCES submissions(submission_id),
    version_number INTEGER NOT NULL,
    submitted_at TEXT NOT NULL,
    content_reference TEXT NOT NULL,
    is_latest_valid INTEGER NOT NULL DEFAULT 1 CHECK (is_latest_valid IN (0,1)),
    UNIQUE (submission_id, version_number)
);

CREATE TABLE extensions (
    extension_id INTEGER PRIMARY KEY,
    assignment_id INTEGER NOT NULL REFERENCES assignments(assignment_id),
    student_id INTEGER NOT NULL REFERENCES users(user_id),
    revised_deadline TEXT NOT NULL,
    reason TEXT NOT NULL CHECK (length(trim(reason)) > 0),
    granted_by INTEGER NOT NULL REFERENCES users(user_id),
    granted_at TEXT NOT NULL,
    UNIQUE (assignment_id, student_id)
);

CREATE TABLE marks (
```

```
    mark_id INTEGER PRIMARY KEY,
    submission_id INTEGER NOT NULL UNIQUE REFERENCES submissions(submission_id),
    raw_mark REAL NOT NULL CHECK (raw_mark >= 0),
    penalty REAL NOT NULL DEFAULT 0 CHECK (penalty >= 0),
    final_mark REAL NOT NULL CHECK (final_mark >= 0),
    feedback TEXT NOT NULL DEFAULT '',
    status TEXT NOT NULL CHECK (
        status IN ('AWAITING_MODERATION','RETURNED','APPROVED','RELEASED')
    ),
    marked_by INTEGER NOT NULL REFERENCES users(user_id),
    marked_at TEXT NOT NULL
);

CREATE TABLE moderation_decisions (
    decision_id INTEGER PRIMARY KEY,
    mark_id INTEGER NOT NULL REFERENCES marks(mark_id),
    hod_user_id INTEGER NOT NULL REFERENCES users(user_id),
    decision TEXT NOT NULL CHECK (decision IN ('APPROVE','AMEND','RETURN')),
    previous_mark REAL,
    new_mark REAL,
    reason TEXT NOT NULL,
    decided_at TEXT NOT NULL
);

CREATE TABLE audit_logs (
    audit_id INTEGER PRIMARY KEY,
    actor_user_id INTEGER NOT NULL REFERENCES users(user_id),
    event_type TEXT NOT NULL,
    entity_type TEXT NOT NULL,
    entity_id INTEGER NOT NULL,
    timestamp TEXT NOT NULL,
    old_value TEXT,
    new_value TEXT,
    reason TEXT NOT NULL
);

CREATE INDEX idx_assignments_course ON assignments(course_id);
CREATE INDEX idx_submissions_student ON submissions(student_id);
CREATE INDEX idx_submissions_assignment ON submissions(assignment_id);
CREATE INDEX idx_marks_status ON marks(status);
CREATE INDEX idx_audit_timestamp ON audit_logs(timestamp);
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
```

6.6 Integrity and transaction constraints

- Enable SQLite foreign-key enforcement for every connection and use parameterised prepared statements.
- Use unique constraints for usernames, course codes, relationship pairs, one submission per student/assignment, and one current mark per submission.
- Application logic must validate raw_mark and final_mark against the linked assignment's maximum mark because a simple SQLite CHECK cannot compare values in another table.
- When a new valid submission version is selected, update the latest-version indicator consistently within a transaction.
- Mark save plus audit, HoD amendment plus moderation decision/audit, penalty recalculation plus audit, and release batch plus audit are atomic operations.
- The application should not expose ordinary update/delete operations for audit rows. Database-level protection can be added if required by the final security design.
- Use one documented timestamp convention, preferably ISO-8601 UTC or a consistently defined local-time convention. The SRS currently assumes the host system clock, so this choice must be documented before implementation.

7. Essential Security and Validation Constraints

Security constraints are included here only where they affect the design of classes, services, database access, or state transitions.

Operation	Administrator	Lecturer	HoD	Student
Manage user accounts / departments / courses	Yes	No	No	No
Assign lecturer / enrol student	Yes	No	No	No
Create assignment for assigned course	No	Yes	No	No
Grant extension for a taught course	No	Yes	No	No
Submit/resubmit own work	No	No	No	Yes
Mark assigned-course submissions after deadline	No	Yes	No	No

Moderate marks in own department	No	No	Yes	No
Release approved marks in own department	No	No	Yes	No
View own released mark and feedback	No	No	No	Yes
View system-wide audit logs	Yes	No	No	No
View audit logs for own department	No	No	Yes	No

- Passwords are never stored in plaintext. Use a recognised password-hashing library and its verification function; Argon2id is the SRS preference.
- All operations check both the authenticated role and relevant resource scope in application services.
- Reject blank required fields, invalid identifiers, nonexistent references, malformed dates, marks outside the permitted range, and extensions without a revised deadline and reason.
- Reject resubmission after the effective deadline or once marking has started; reject release requests containing any unapproved mark.
- On persistence failure, roll back the operation, report a safe error, and do not display a success message.

8. Requirements-to-Design Traceability

This summary connects the principal requirements to the design artefacts and shows how the design can be checked. The identifiers must be reconciled with the final approved Week 2 SRS.

Requirement(s)	Design artefact	Verification focus
FR-01, FR-02; NFR-03, NFR-04	AuthenticationService, AuthorizationService, users table	Valid/invalid login, role-negative tests, stored password hashes.
FR-03 to FR-07	User/Course services, users/departments/courses and relationship tables	Account, course, ownership, allocation, and enrolment rules.
FR-08, FR-09, FR-15	SubmissionService, SubmissionLifecyclePolicy, LatePenaltyPolicy, submission_versions	Deadline boundaries, late penalties, resubmission, and lifecycle transitions.
FR-10	ExtensionService, Extension class/table	Mandatory reason, revised deadline, and access scope.
FR-11	MarkingService, Mark class/table	Mark range, deadline guard, feedback storage, and hidden-until-release behaviour.
FR-12, FR-13	ModerationService, ReleaseService, moderation_decisions and marks	Approve/amend/return flows and prevention of premature visibility.
FR-14, FR-18; NFR-06	AuditService, audit_logs, permission policy	Audit completeness, old/new values, scoped access, and append-only application behaviour.
FR-16; NFR-05	Validation, SQL constraints, TransactionManager	Invalid input, foreign-key/check constraints, rollback, and atomicity.
NFR-01, NFR-02	SQLite repositories and local deployment	Performance target and core workflows with network disconnected.

9. Design Review Items

The following decisions must be resolved against the approved SRS before implementation. They are listed here to prevent the design from silently inventing policy.

- For submissions more than three days late, confirm whether the system retains submission content or only attempt metadata while assigning the specified zero outcome.
- Confirm whether an extension can be changed after it is granted and whether multiple extensions are allowed.
- Confirm the account-lockout threshold, duration, and reset behaviour.
- Confirm the timestamp/time-zone convention and how the host system clock is handled.
- Confirm the workflow after a mark is returned for correction, including whether re-marking is permitted and what status follows.
- Review diagram labels, class names, table names, and lifecycle states against the final approved SRS.

Conclusion

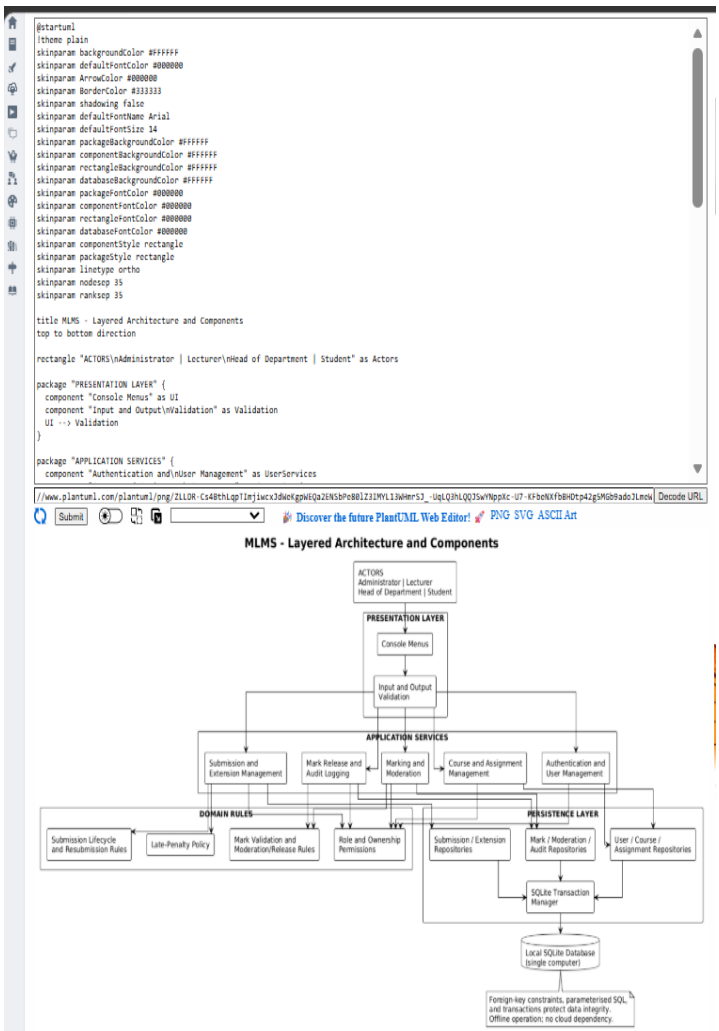
This design proposes a layered offline architecture, a UML domain model, explicit submission and moderation workflows, replaceable policy and persistence abstractions, and a relational SQLite schema intended to satisfy 3NF. The central invariant is that students cannot view marks or feedback until a mark has been approved and released, and mark-related state changes are recorded with their audit events transactionally. The unresolved policy decisions in Section 9 must be confirmed before implementation.

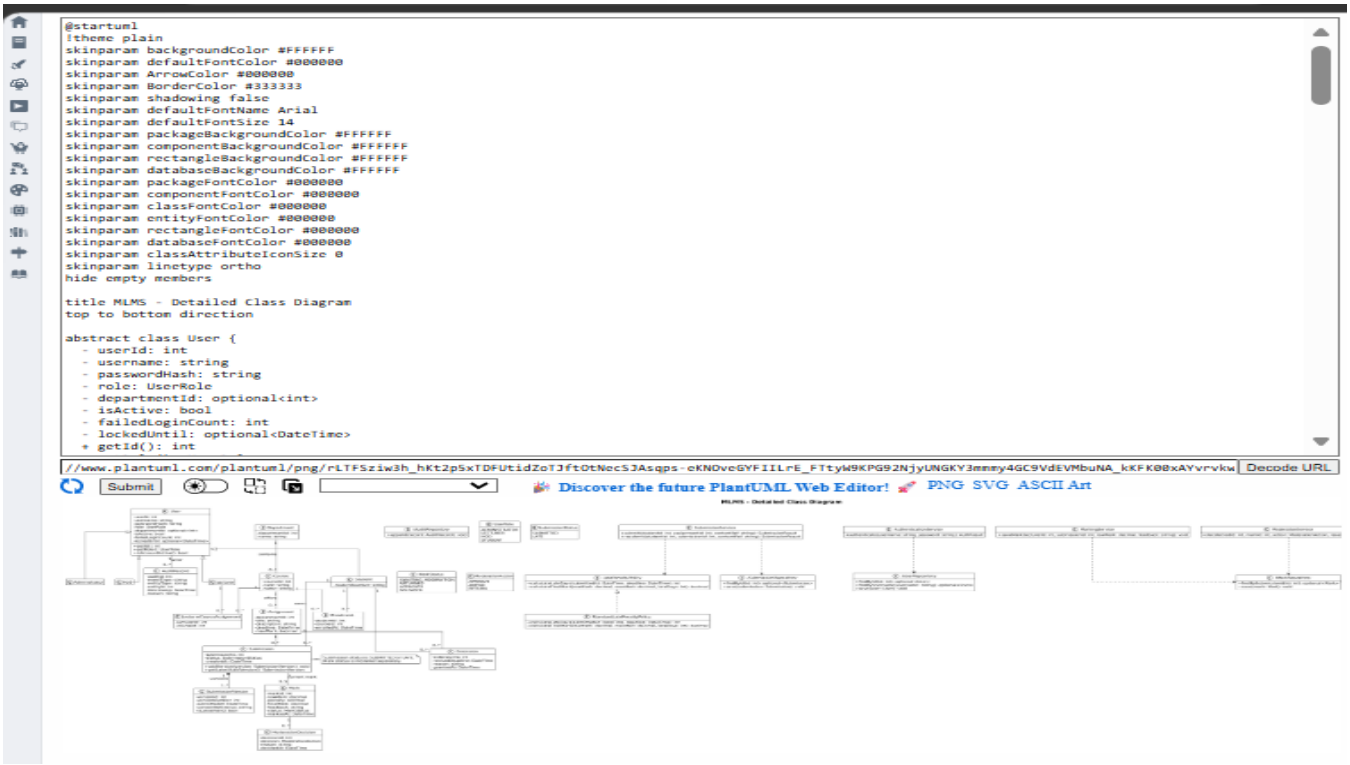
References

- Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO.
- Molapo, S. (2026). CS4433 Project Management Plan. Week 1 submission
- Molapo, S. (2026). CS4433 Requirements Engineering Week 2 submission,

Proof of how the diagrams were made

1.


@startuml
!theme plain
skinparam backgroundColor #FFFFFF
skinparam defaultFontColor #000000
skinparam ArrowColor #000000
skinparam BorderColor #333333
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 14
skinparam packageBackgroundColor #FFFFFF
skinparam componentBackgroundColor #FFFFFF
skinparam rectangleBackgroundColor #FFFFFF
skinparam databaseBackgroundColor #FFFFFF
skinparam packageFontColor #000000
skinparam componentFontColor #000000
skinparam rectangleFontColor #000000
skinparam databaseFontColor #000000
skinparam componentStyle rectangle
skinparam packageStyle rectangle
skinparam linetype ortho
skinparam nodesep 35
skinparam ranksep 35
title MLMS - Layered Architecture and Components
top to bottom direction
rectangle "ACTORS\nAdministrator | Lecturer\nHead of Department | Student" as Actors
package "PRESENTATION LAYER" {
component "Console Menus" as UI
component "Input and Output\nValidation" as Validation
UI --> Validation
}
package "APPLICATION SERVICES" {
component "Authentication and\nUser Management" as UserServices
Decode URL
Submit
Discover the future PlantUML Web Editor!
PNG SVG ASCII Art
MLMS - Layered Architecture and Components
ACTORS
Administrator | Lecturer
Head of Department | Student
PRESENTATION LAYER
Console Menus
Input and Output
Validation
APPLICATION SERVICES
Submission and
Extension Management
Mark Release and
Audit Logging
Marking and
Moderation
Course and Assignment
Management
Authentication and
User Management
DOMAIN RULES
PERSISTENCE LAYER
Submission Lifecycle
and Resubmission Rules
Late-Penalty Policy
Mark Validation and
Moderation/Release Rules
Role and Ownership
Permissions
Submission / Extension
Repositories
Mark / Moderation /
Audit Repositories
User / Course /
Assignment Repositories
SQLite Transaction
Manager
Local SQLite Database
(single computer)
Foreign-key constraints, parameterised SQL,
and transactions protect data integrity.
Offline operation; no cloud dependency.


@startuml
!theme plain
skinparam backgroundColor #FFFFFF
skinparam defaultFontColor #000000
skinparam ArrowColor #000000
skinparam BorderColor #333333
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 14
skinparam packageBackgroundColor #FFFFFF
skinparam componentBackgroundColor #FFFFFF
skinparam rectangleBackgroundColor #FFFFFF
skinparam databaseBackgroundColor #FFFFFF
skinparam packageFontColor #000000
skinparam componentFontColor #000000
skinparam classFontColor #000000
skinparam entityFontColor #000000
skinparam rectangleFontColor #000000
skinparam databaseFontColor #000000
skinparam classAttributeIconSize 0
skinparam linetype ortho
hide empty members
title MLMS - Detailed Class Diagram
top to bottom direction
abstract class User {
- userId: int
- username: string
- passwordHash: string
- role: UserRole
- departmentId: optional<int>
- isActive: bool
- failedLoginCount: int
- lockedUntil: optional<DateTime>
+ getId(): int
Decode URL
Submit
Discover the future PlantUML Web Editor!
PNG SVG ASCII Art

3.

```
!theme plain
skinparam backgroundColor #FFFFFF
skinparam defaultFontColor #000000
skinparam ArrowColor #000000
skinparam BorderColor #333333
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 14
skinparam stateBackgroundColor #FFFFFF
skinparam stateBorderColor #333333
skinparam stateFontColor #000000

title MLMS - Submission and Mark Lifecycle
top to bottom direction

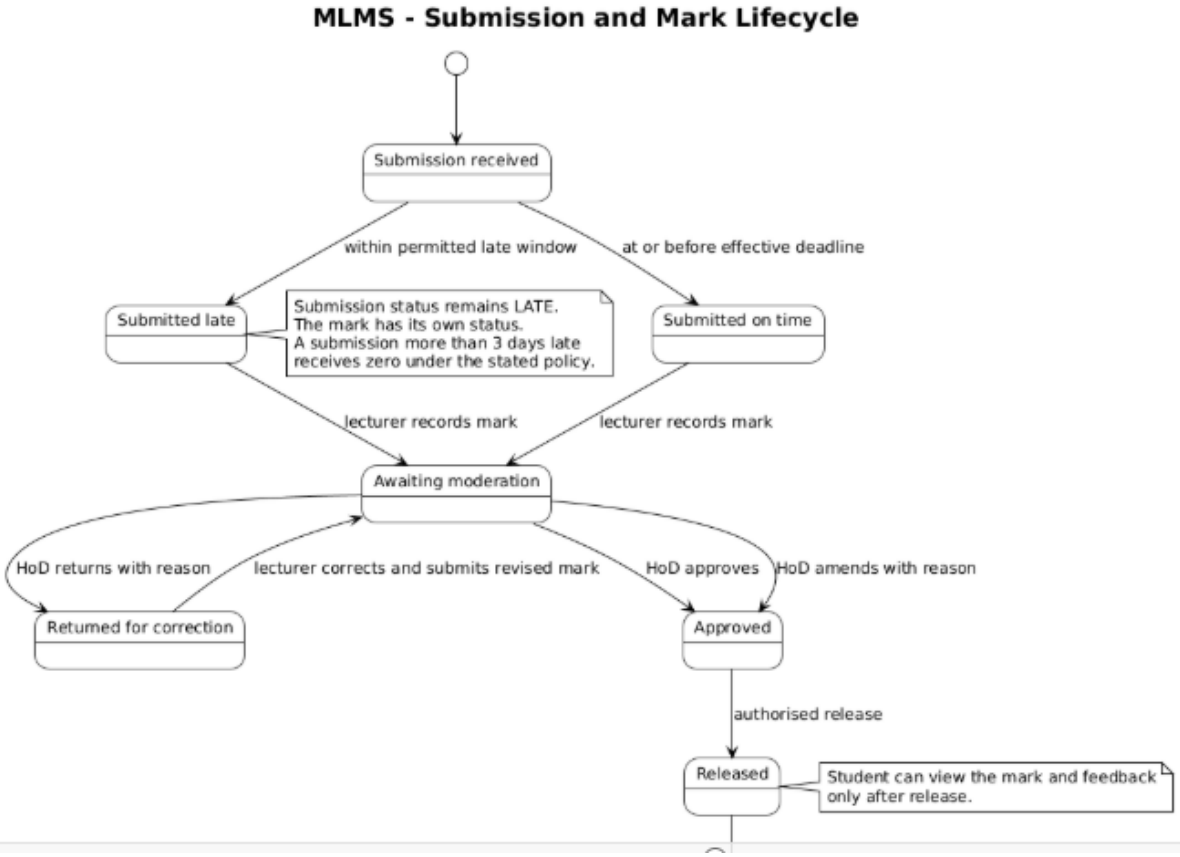
state "Submission received" as Received
state "Submitted on time" as Submitted
state "Submitted late" as Late
state "Awaiting moderation" as Awaiting
state "Returned for correction" as Returned
state "Approved" as Approved
state "Released" as Released

[*] --> Received
Received --> Submitted : at or before effective deadline
Received --> Late : within permitted late window
Submitted --> Awaiting : lecturer records mark
Late --> Awaiting : lecturer records mark
Awaiting --> Returned : HoD returns with reason
Returned --> Awaiting : lecturer corrects and submits revised mark
Awaiting --> Approved : HoD approves
Awaiting --> Approved : HoD amends with reason
Approved --> Released : authorised release
Released --> [*]
```

//www.plantuml.com/plantuml/png/ZPJ1RXGn38RlUGfJRaYjGEN40NKBL1nQ3jtU40VlnDE9DXEF74z7swSdarrhfxG5vXJ7dvt_Tx9xaHL5npwwOzfHJp143CdbRKW32lQmmMF Decode URL

Submit

Discover the future PlantUML Web Editor! PNG SVG ASCII Art



4.

https://www.plantuml.com/plantuml/uml/lLR1RXit4BtlLx3efIqie4Lx4e8WIIstGHoZSMoWjyLeEP88Sya5Xoj7jVll9TS3PTQxui45eel5zvxdpGo7fDvGX13xrbRVnHssg3eBnbLqPrm70LgrXkPk6tpl...

```
@startuml
!theme plain
skinparam backgroundColor #FFFFFF
skinparam defaultFontColor #000000
skinparam ArrowColor #000000
skinparam BorderColor #333333
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 14
skinparam entityBackgroundColor #FFFFFF
skinparam entityBorderColor #333333
skinparam entityFontColor #000000
skinparam linetype ortho

title MLMS - Logical Database ERD
hide circle

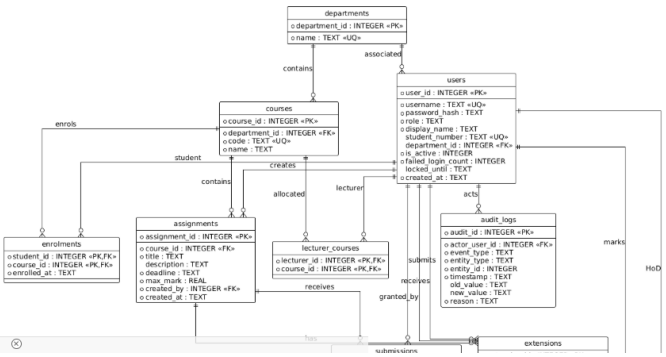
entity departments {
  * department_id : INTEGER <<PK>>
  --
  * name : TEXT <<UQ>>
}

entity users {
  * user_id : INTEGER <<PK>>
  --
  * username : TEXT <<UQ>>
  * password_hash : TEXT
  * role : TEXT
  * display_name : TEXT
  student_number : TEXT <<UQ>>
  department_id : INTEGER <<FK>>
  * is_active : INTEGER
  * failed_login_count : INTEGER
  locked_until : TEXT
  * created_at : TEXT
```

https://www.plantuml.com/plantuml/png/lLR1RXit4BtlLx3efIqie4Lx4e8WIIstGHoZSMoWjyLeEP88SyaSXoj7jVll9TS3PTQxui45eel5zvxdpGo7fDvGX13xrbRVnHssg3eBnbLqPrm7dLgrXkPk6tplzt_U-g2-lnW-Gg5nBxsDSzv5bleyV8JaRGZ-_XJvpWUDWT9VXuzWQDVQtnkt1vKmnXFE-Lz1YCZSWv_dFvXzKl_mcI7JHnCExGdSiLqoHDQNwhN4OJnqg7-BELrNIMrGVBpz-LWjrwRUcBQlE8SBQ2DMg-hpQGOsgCQ6nMSLF1afZYXRRj2BrKZdFraqIgezL1I Decode URL

Submit | Discover the future PlantUML Web Editor! | PNG SVG ASCII Art

MLMS - Logical Database ERD



https://www.plantuml.com/plantuml/uml/ZLHFRzfE4BtxKyn...

```
@startuml
!theme plain
skinparam backgroundColor #FFFFFF
skinparam defaultFontColor #000000
skinparam ArrowColor #000000
skinparam BorderColor #333333
skinparam shadowing false
skinparam defaultFontName Arial
skinparam defaultFontSize 14
skinparam activityBackgroundColor #FFFFFF
skinparam activityBorderColor #333333
skinparam activityDiamondBackgroundColor #FFFFFF
skinparam activityDiamondBorderColor #333333
skinparam activityFontColor #000000

title MLMS - Submission to Released Mark

|Student|
start
:Sign in;
:Select assignment;
:Submit work or request resubmission;

|System|
:Authenticate and authorise;
:Load assignment and extension;
:Calculate effective deadline;
if (Submission eligible?) then (Yes)
  :Validate submission rules;
  :Save submission version;
  :Record audit event;
  if (Within effective deadline?) then (Yes)
    :Set submission status SUBMITTED;
  else (No)
    :Set submission status LATE;
  endif
```

//www.plantuml.com/plantuml/png/ZLHFRzfE4BtxKynFlqjoYDIgFS6XSG9faVWJOPAcfshZ7SEKzQuxkuPIyU6xQnko4XJ5XoZol7blvht1jzO9TdMfal_S2ak4IWdIYLsJhW1 Decode URL

Submit

Discover the future PlantUML Web Editor! PNG SVG ASCII Art

MLMS - Submission to Released Mark

